```
"""Q-learning for a 4x4 maze, compared with a random-action baseline."""
import csv
import random
import statistics

SIZE = 4
START = (0, 0)
GOAL = (3, 3)
BLOCKED = {(1, 1), (2, 1)}
ACTIONS = ((0, 1), (0, -1), (1, 0), (-1, 0))  # right, left, down, up

ALPHA = 0.20
GAMMA = 0.95
EPISODES = 1000
MAX_STEPS = 50
SEED = 7
EVALUATION_EPISODES = 200


def step(state, action):
    """Apply one action and return next_state, reward, terminal."""
    row, col = state
    dr, dc = action
    next_state = (row + dr, col + dc)

    if not (0 <= next_state[0] < SIZE and 0 <= next_state[1] < SIZE):
        return state, -5, False
    if next_state in BLOCKED:
        return state, -5, False
    if next_state == GOAL:
        return next_state, 10, True
    return next_state, -1, False


def random_episode(rng):
    """Run one random episode; return total reward and whether it succeeded."""
    state = START
    total_reward = 0

    for _ in range(MAX_STEPS):
        action = rng.choice(ACTIONS)
        state, reward, done = step(state, action)
        total_reward += reward
        if done:
            return total_reward, True

    return total_reward, False


def evaluate_q_policy(q, rng):
```

```
    """Evaluate the greedy learned policy over 200 episodes."""
    successes = 0
    successful_lengths = []

    for _ in range(EVALUATION_EPISODES):
        state = START

        for move in range(1, MAX_STEPS + 1):
            values = q.get(state, [0.0] * len(ACTIONS))
            best_value = max(values)
            best_actions = [i for i, value in enumerate(values)
                            if value == best_value]
            action = rng.choice(best_actions)
            state, _, done = step(state, ACTIONS[action])

            if done:
                successes += 1
                successful_lengths.append(move)
                break

    return successes, successful_lengths


def evaluate_random_policy(rng):
    """Evaluate random actions over 200 episodes."""
    successes = 0
    successful_lengths = []

    for _ in range(EVALUATION_EPISODES):
        state = START

        for move in range(1, MAX_STEPS + 1):
            action = rng.choice(ACTIONS)
            state, _, done = step(state, action)

            if done:
                successes += 1
                successful_lengths.append(move)
                break

    return successes, successful_lengths


def main():
    rng = random.Random(SEED)
    q_values = {}
    training_results = []

    # Train tabular Q-learning with epsilon-greedy exploration.
    epsilon = 0.20
```

```
for episode in range(1, EPISODES + 1):
    state = START
    total_reward = 0
    success = 0

    for _ in range(MAX_STEPS):
        q_values.setdefault(state, [0.0] * len(ACTIONS))

        if rng.random() < epsilon:
            action_index = rng.randrange(len(ACTIONS))
        else:
            best_value = max(q_values[state])
            best_actions = [i for i, value in enumerate(q_values[state])
                            if value == best_value]
            action_index = rng.choice(best_actions)

        next_state, reward, done = step(state, ACTIONS[action_index])
        q_values.setdefault(next_state, [0.0] * len(ACTIONS))

        # No future value is added after the terminal goal state.
        future_value = 0 if done else max(q_values[next_state])
        old_value = q_values[state][action_index]
        q_values[state][action_index] = old_value + ALPHA * (
            reward + GAMMA * future_value - old_value
        )

        state = next_state
        total_reward += reward

        if done:
            success = 1
            break

    training_results.append((episode, total_reward, success))

# Make learning-curve data: mean return in consecutive 50-episode windows.
curve_rows = []
for start in range(0, EPISODES, 50):
    window = training_results[start:start + 50]
    random_returns = [random_episode(rng)[0] for _ in window]
    curve_rows.append((
        window[-1][0],
        statistics.mean(row[1] for row in window),
        statistics.mean(random_returns),
    ))

with open("learning_curve_data.csv", "w", newline="", encoding="utf-8") as file:
    writer = csv.writer(file)
    writer.writerow([
        "episode",
```

```
            "q_learning_mean_return_50ep",
            "random_mean_return_50ep",
        ])
        writer.writerows(curve_rows)

    # Evaluate both policies on 200 new episodes each.
    q_successes, q_lengths = evaluate_q_policy(q_values, rng)
    random_successes, random_lengths = evaluate_random_policy(rng)

    print("Learning-curve data saved to learning_curve_data.csv")
    print("Evaluation episodes per policy:", EVALUATION_EPISODES)
    print(f"Q-learning success rate: {q_successes / 2:.1f}%")
    if q_lengths:
        print(f"Q-learning mean moves when successful: "
              f"{statistics.mean(q_lengths):.1f}")
    else:
        print("Q-learning had no successful episodes.")

    print(f"Random-agent success rate: {random_successes / 2:.1f}%")
    if random_lengths:
        print(f"Random-agent mean moves when successful: "
              f"{statistics.mean(random_lengths):.1f}")
    else:
        print("The random agent had no successful episodes.")


if __name__ == "__main__":
    main()
```